Alu CFG/Ativa (ELO) RAQUEL BELCHIOR FACANHA NOGUEIRA (SE/3),
Alu CFG/Ativa MATEUS DA SILVA MALDONADO (SE/6),
Alu CFG/Ativa ESTEVAO JOHNATAS RIBEIRO BATISTA (SE/6),
Alu CFG/Ativa RAFAEL DE FIGUEIREDO SCHUINKI (SE/9)

Relatório de Andamento: Calibração entre Sensor de Profundidade e Projetor para o Sistema de Caixão de Areia com Realidade Aumentada

Rio de Janeiro – RJ

2 de março de 2026

SUMÁRIO

Sumário 1

1 INTRODUÇÃO 3

2 O SISTEMA DE CAIXÃO DE AREIA COM REALIDADE AUMENTADA 4
2.1 Conceito e Estado da Arte 4
2.2 O Problema de Calibração 4

3 FUNDAMENTOS: TEORIA DOS MÍNIMOS QUADRADOS . . . 6
3.1 Motivação e Critério de Otimização 6
3.2 Exemplo Introdutório: Estimação da Gravidade 6
3.3 Formulação Matricial e a Pseudo-Inversa 7

4 AJUSTE POLINOMIAL 2D E EXTENSÃO PARA 3D 8
4.1 Ajuste Polinomial em 2D 8
4.2 Extensão para 3D: Ajuste de Plano 9

5 CALIBRAÇÃO KINECT–PROJETOR 10
5.1 Modelo de Mapeamento Quadrático 10
5.2 Coleta de Dados de Calibração 10
5.3 Avaliação da Qualidade: Erro de Reprojeção 11

6 FORMULAÇÃO RECURSIVA 13
6.1 Motivação 13
6.2 Equações Recursivas 13

7 PRÓXIMOS PASSOS 15

REFERÊNCIAS 16

Alu CFG/Ativa (ELO) RAQUEL BELCHIOR FACANHA NOGUEIRA (SE/3),
Alu CFG/Ativa MATEUS DA SILVA MALDONADO (SE/6),
Alu CFG/Ativa ESTEVAO JOHNATAS RIBEIRO BATISTA (SE/6),
Alu CFG/Ativa RAFAEL DE FIGUEIREDO SCHUINKI (SE/9)

Relatório de Andamento: Calibração entre Sensor de Profundidade e Projetor para o Sistema de Caixão de Areia com Realidade Aumentada

Relatório parcial de Projeto Final de Curso apresentado aos orientadores como registro do progresso nos estudos de calibração do sistema de Caixão de Areia com Realidade Aumentada, com foco na Teoria dos Mínimos Quadrados aplicada ao mapeamento entre o sensor Kinect e o projetor.

INSTITUTO MILITAR DE ENGENHARIA - IME

Orientador: Prof. DANIEL RODRIGUES DOS SANTOS,
TC LEONARDO ASSUMPCAO MOREIRA,
TC BRUNO EDUARDO MADEIRA,
Cap GABRIEL DA CRUZ FONTENELLE

Rio de Janeiro – RJ
2 de março de 2026

1 INTRODUÇÃO

No contexto do preparo e da emissão de ordens no nível tático, o reconhecimento e a representação do terreno constituem etapas fundamentais do planejamento militar. Nesse sentido, o caixão de areia é uma ferramenta tradicional que permite ao comandante visualizar e comunicar de forma concreta o posicionamento de tropas, eixos de deslocamento e pontos de interesse sobre um modelo físico tridimensional do terreno.

O presente Projeto Final de Curso, solicitado pela Academia Militar das Agulhas Negras (AMAN), propõe modernizar essa ferramenta integrando tecnologias de realidade aumentada, sensoriamento de profundidade e fontes de dados geoespaciais abertas. Sob esse viés, o sistema projetado é capaz de: ler a topografia modelada na areia por meio do sensor Kinect; comparar o modelo gerado com o modelo digital de terreno de cartas topográficas de referência; fornecer *feedback* visual sobre a fidelidade do modelo físico; e integrar fontes abertas como Google Earth e OpenStreetMap para navegação a partir de pontos indicados no caixão [1].

Este relatório documenta especificamente o estudo e o desenvolvimento da **calibração entre o sensor Kinect e o projetor**. Essa etapa é necessária para que a imagem projetada sobre a areia coincida com precisão com o relevo detectado pelo sensor. Para isso, o método adotado baseia-se na **Teoria dos Mínimos Quadrados**.

2 O SISTEMA DE CAIXÃO DE AREIA COM REALIDADE AUMENTADA

2.1 Conceito e Estado da Arte

O conceito de caixão de areia com realidade aumentada (*augmented reality sandbox*) foi desenvolvido por Oliver Kreylos no *W.M. Keck Center for Active Visualization in the Earth Sciences* (KeckCAVES) da Universidade da Califórnia em Davis, resultando no software de código aberto SARndbox [3]. Esse sistema original escaneia a superfície da areia com um sensor Kinect e projeta sobre ela, em tempo real, um mapa topográfico com curvas de nível, sombreamento de relevo e simulação de fluxo d'água [4].

Desde sua publicação, o projeto tornou-se referência mundial e foi replicado em museus, escolas e centros de ciência. Por exemplo, versões como o *Magic-Sand* [6] portaram o conceito para múltiplas plataformas (Linux, Windows e macOS) com interface simplificada. Além disso, guias DIY (*Do It Yourself*) detalhados tornaram a montagem acessível a instituições com recursos limitados [5].

O presente projeto distingue-se dessas implementações ao adicionar: comparação automática com MDT de cartas topográficas reais; feedback quantitativo de fidelidade do modelo físico; e integração com navegação geoespacial — funcionalidades com aplicação direta no contexto do planejamento tático militar.

2.2 O Problema de Calibração

Para que a imagem projetada coincida com precisão com o relevo detectado pelo Kinect, os dois dispositivos precisam estar **calibrados entre si**. Pois, o Kinect e o projetor estão fisicamente posicionados em locais diferentes e com ângulos distintos, o que significa que um mesmo ponto no espaço real corresponde a coordenadas diferentes em cada dispositivo.

Nessa perspectiva, o Kinect reporta cada ponto detectado como uma tripla (u_k, v_k, d_k), onde u_k e v_k são as coordenadas em pixels na imagem do sensor e d_k é a profundidade em milímetros. Ademais, o projetor trabalha com coordenadas 2D (u_p, v_p). Desse modo, o problema de calibração consiste em determinar as funções de mapeamento:

$$u_p = f(u_k, v_k, d_k), \qquad v_p = g(u_k, v_k, d_k) \tag{2.1}$$

Esse mapeamento é não-trivial pois envolve perspectivas diferentes, distorção de lente

e a influência da profundidade na posição projetada. Apesar disso, o SARndbox resolve esse problema com uma utilitário próprio (`CalibrateProjector`), que coleta pontos de correspondência e estima uma matriz de transformação [4]. Neste projeto, o mesmo problema é abordado, porém com o método dos Mínimos Quadrados, permitindo o uso de um **modelo quadrático** mais flexível.

3 FUNDAMENTOS: TEORIA DOS MÍNIMOS QUADRADOS

3.1 Motivação e Critério de Otimização

O método dos mínimos quadrados, desenvolvido por Gauss (1795), encontra os parâmetros de um modelo matemático que melhor se ajusta a um conjunto de medidas com erros e ruídos [2]. O critério adotado desse método é minimizar a **soma dos quadrados dos erros** entre os valores medidos y_i e os valores estimados pelo modelo $\hat{y}_i$:

$$J = \sum_{i=1}^{k} e_i^2 = \sum_{i=1}^{k} (y_i - \hat{y}_i)^2 \tag{3.1}$$

A escolha do quadrado — e não do valor absoluto, por exemplo — é conveniente por ser diferenciável e produzir um único mínimo global para sistemas bem-postos.

3.2 Exemplo Introdutório: Estimação da Gravidade

Para fixar o conceito, considera-se o exemplo clássico dos slides [2]: um instrumento mede a velocidade de um corpo em queda livre em instantes t_i, com erros instrumentais. O modelo teórico é $v_i = g\,t_i$. Aplicando o critério (3.1):

$$J = \sum_{i=1}^{N} (v_i - \hat{g}\,t_i)^2 \tag{3.2}$$

Derivando em relação a $\hat{g}$ e igualando a zero, obtém-se a estimativa ótima:

$$\hat{g} = \frac{\sum_{i=1}^{N} v_i t_i}{\sum_{i=1}^{N} t_i^2} \tag{3.3}$$

A Figura 1 (gerada pelo Script 1 MATLAB) ilustra o resultado.

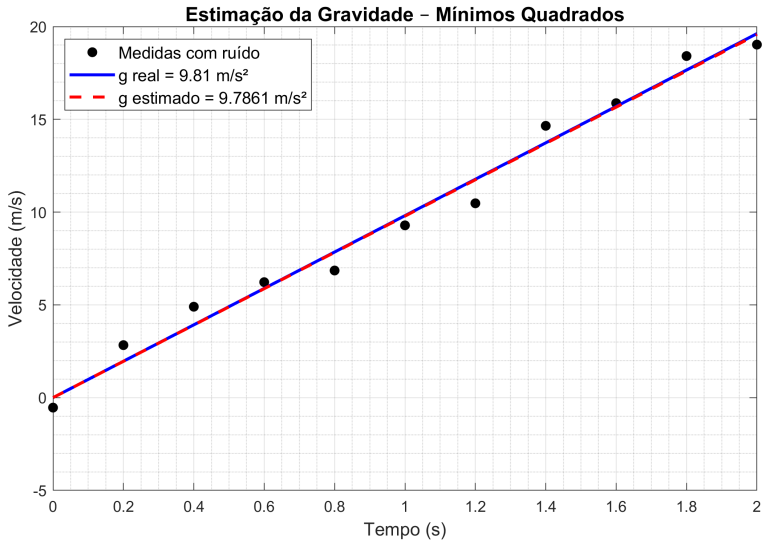

Figura 1 – Estimação da aceleração da gravidade pelo método dos mínimos quadrados. Pontos: medidas com ruído gaussiano. Linha azul: valor verdadeiro $g = 9{,}81$ m/s². Linha vermelha: estimativa.

Fonte: Gerada pelo autor via Script MATLAB 1.

3.3 Formulação Matricial e a Pseudo-Inversa

Generalizando para n parâmetros desconhecidos e k medidas, o sistema é escrito na forma matricial (com $k > n$, sistema sobredeterminado):

$$\underbrace{\mathbf{y}}_{k\times 1} = \underbrace{\mathbf{X}}_{k\times n}\underbrace{\hat{\mathbf{a}}}_{n\times 1} + \underbrace{\mathbf{e}}_{k\times 1} \tag{3.4}$$

O critério $J = \mathbf{e}^T\mathbf{e}$ é minimizado derivando em relação a $\hat{\mathbf{a}}$:

$$\frac{\partial J}{\partial \hat{\mathbf{a}}} = -2\mathbf{X}^T\mathbf{y} + 2\mathbf{X}^T\mathbf{X}\hat{\mathbf{a}} = \mathbf{0} \tag{3.5}$$

Resultando na solução pela **pseudo-inversa**:

$$\boxed{\hat{\mathbf{a}} = \left(\mathbf{X}^T\mathbf{X}\right)^{-1}\mathbf{X}^T\mathbf{y}} \tag{3.6}$$

O termo $(\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T$ é chamado de pseudo-inversa porque faz o papel de uma inversa em situações onde $\mathbf{X}$ não é quadrada, o que ocorre sempre que há mais medidas do que parâmetros. No MATLAB:

```
a_hat = (X' * X) \ (X' * y);
```

Listing 3.1 – Cálculo da pseudo-inversa no MATLAB.

4 AJUSTE POLINOMIAL 2D E EXTENSÃO PARA 3D

4.1 Ajuste Polinomial em 2D

Dado um conjunto de medidas (z_i, y_i), deseja-se ajustar o polinômio de grau n:

$$P(z) = a_0 + a_1 z + a_2 z^2 + \cdots + a_n z^n \tag{4.1}$$

Cada medida gera uma linha da **matriz de Vandermonde**:

$$\mathbf{X} = \begin{bmatrix} 1 & z_1 & z_1^2 & \cdots & z_1^n \\ 1 & z_2 & z_2^2 & \cdots & z_2^n \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & z_k & z_k^2 & \cdots & z_k^n \end{bmatrix} \tag{4.2}$$

A pseudo-inversa (3.6) retorna $\hat{\mathbf{a}} = [a_0, a_1, \ldots, a_n]^T$. A Figura 2 mostra os resultados para diferentes graus sobre os dados do exemplo dos slides.

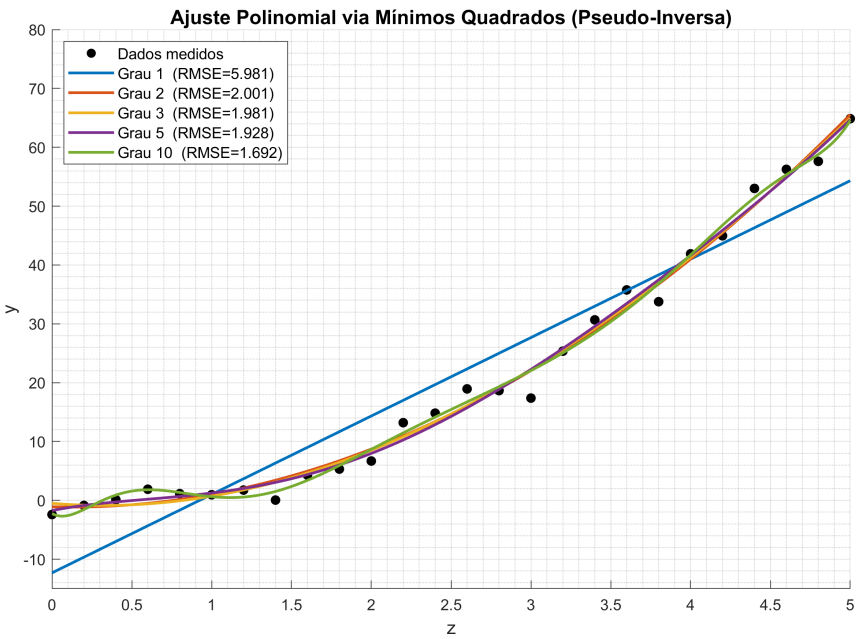

Figura 2 – Ajuste polinomial com graus 1, 2, 3, 5 e 10. Graus elevados reduzem o RMSE nos dados de treinamento, mas podem apresentar *overfitting*.

Fonte: Gerada pelo autor via Script MATLAB 2.

4.2 Extensão para 3D: Ajuste de Plano

A generalização para o espaço 3D é direta: basta adicionar a segunda variável independente como coluna em $\mathbf{X}$. Assim, para ajustar um plano $z = a_0 + a_1 x + a_2 y$ a pontos (x_i, y_i, z_i):

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 & y_1 \\ 1 & x_2 & y_2 \\ \vdots & \vdots & \vdots \\ 1 & x_k & y_k \end{bmatrix}, \qquad \mathbf{y} = \begin{bmatrix} z_1 \\ z_2 \\ \vdots \\ z_k \end{bmatrix} \tag{4.3}$$

A fórmula (3.6) é aplicada sem nenhuma alteração. A Figura 3 ilustra o resultado.

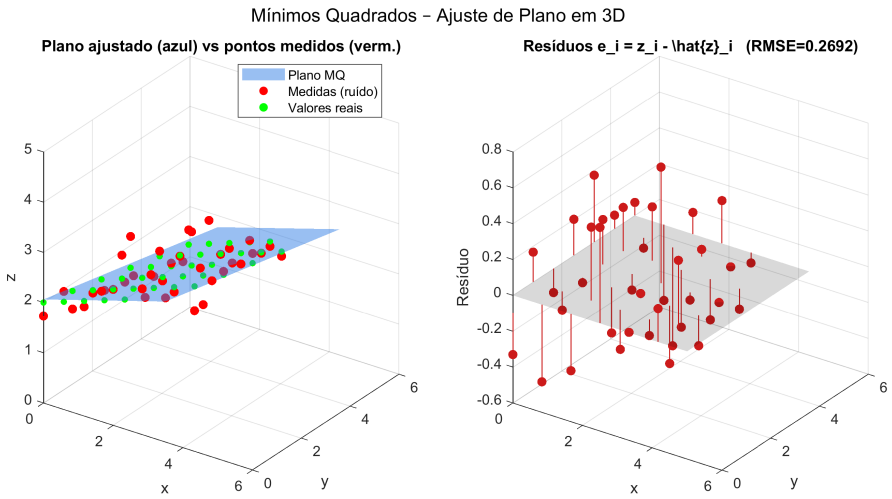

Figura 3 – Ajuste de plano a pontos 3D com ruído gaussiano ($\sigma = 0{,}3$ mm). Superfície azul: plano estimado. Pontos vermelhos: medidas. Pontos verdes: valores reais.

Fonte: Gerada pelo autor via Script MATLAB 3.

A tabela abaixo resume a correspondência estrutural entre os casos:

Tabela 1 – Comparativo entre os casos de ajuste.

Caso	**Linha de X**	**Parâmetros**
Reta (2D)	$[1,\ z_i]$	a_0, a_1
Quadrático (2D)	$[1,\ z_i,\ z_i^2]$	a_0, a_1, a_2
Plano (3D)	$[1,\ x_i,\ y_i]$	a_0, a_1, a_2
Quadrático (3D)	$[1,\ x_i,\ y_i,\ x_i^2,\ y_i^2,\ x_i y_i]$	$a_0, \ldots, a_5$

Fonte: Elaborado pela autora.

5 CALIBRAÇÃO KINECT–PROJETOR

5.1 Modelo de Mapeamento Quadrático

Para capturar as não-linearidades ópticas do sistema (distorção de lente, perspectiva, variação com profundidade), adota-se um **modelo quadrático completo** com 10 parâmetros por coordenada de saída:

$$\begin{aligned} u_p \approx\ & a_0 + a_1 u_k + a_2 v_k + a_3 d_k \\ & + a_4 u_k^2 + a_5 v_k^2 + a_6 d_k^2 \\ & + a_7\, u_k v_k + a_8\, u_k d_k + a_9\, v_k d_k \end{aligned} \tag{5.1}$$

A matriz de regressão, com k pares de calibração coletados, é:

$$\mathbf{X} = \begin{bmatrix} 1 & u_{k_1} & v_{k_1} & d_{k_1} & u_{k_1}^2 & v_{k_1}^2 & d_{k_1}^2 & u_{k_1} v_{k_1} & u_{k_1} d_{k_1} & v_{k_1} d_{k_1} \\ \vdots & & & & & \vdots & & & & \vdots \\ 1 & u_{k_N} & v_{k_N} & d_{k_N} & u_{k_N}^2 & v_{k_N}^2 & d_{k_N}^2 & u_{k_N} v_{k_N} & u_{k_N} d_{k_N} & v_{k_N} d_{k_N} \end{bmatrix} \tag{5.2}$$

dimensão $k \times 10$, exigindo $k \geq 10$ pares. Resolvem-se dois sistemas independentes com a pseudo-inversa:

$$\hat{\mathbf{a}}_u = \left(\mathbf{X}^T \mathbf{X}\right)^{-1} \mathbf{X}^T \mathbf{u}_p, \qquad \hat{\mathbf{a}}_v = \left(\mathbf{X}^T \mathbf{X}\right)^{-1} \mathbf{X}^T \mathbf{v}_p \tag{5.3}$$

5.2 Coleta de Dados de Calibração

O procedimento de coleta é análogo ao empregado pelo utilitário `CalibrateProjector` do SARndbox [4]:

1. O projetor exibe um tabuleiro de xadrez sobre a areia nivelada. Como a imagem é gerada internamente, as coordenadas (u_p, v_p) de cada canto são exatamente conhecidas;
2. O Kinect captura a cena e o OpenCV detecta automaticamente os cantos na imagem do sensor, fornecendo (u_k, v_k, d_k);
3. O processo é repetido com a areia em diferentes alturas, para que o modelo aprenda o efeito da variação de profundidade;
4. Os pares formam a matriz $\mathbf{X}$ e a pseudo-inversa estima $\hat{\mathbf{a}}_u$ e $\hat{\mathbf{a}}_v$.

5.3 Avaliação da Qualidade: Erro de Reprojeção

A qualidade da calibração é medida pelo **erro de reprojeção** — a distância em pixels entre o ponto projetado real e o estimado pelo modelo:

$$\varepsilon_i = \sqrt{(u_{p,i} - \hat{u}_{p,i})^2 + (v_{p,i} - \hat{v}_{p,i})^2} \tag{5.4}$$

$$\text{RMSE} = \sqrt{\frac{1}{k}\sum_{i=1}^{k} \varepsilon_i^2} \tag{5.5}$$

As Figuras 4 e 5 mostram os resultados comparativos entre os modelos linear e quadrático para dados simulados.

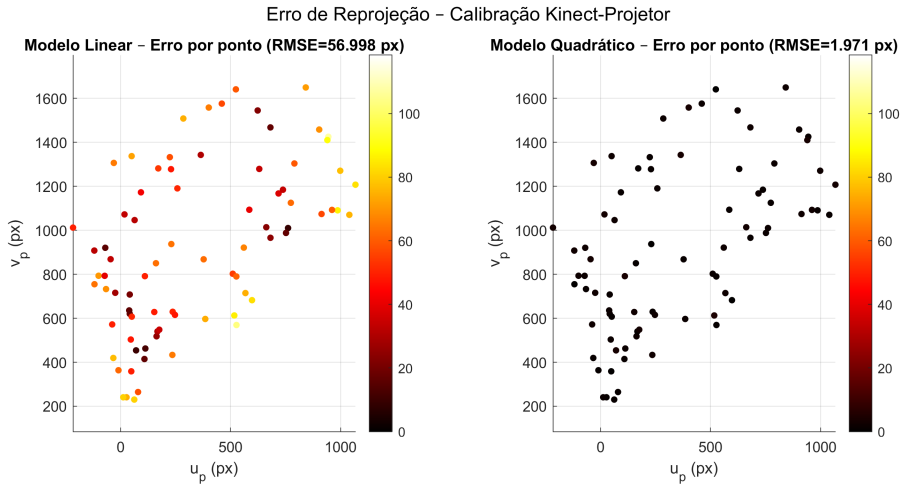

Figura 4 – Mapa de erro de reprojeção por ponto (pixels) para os modelos linear (esq.) e quadrático (dir.). Escala de cores idêntica para comparação direta.

Fonte: Gerada pela autora via Script MATLAB 3.

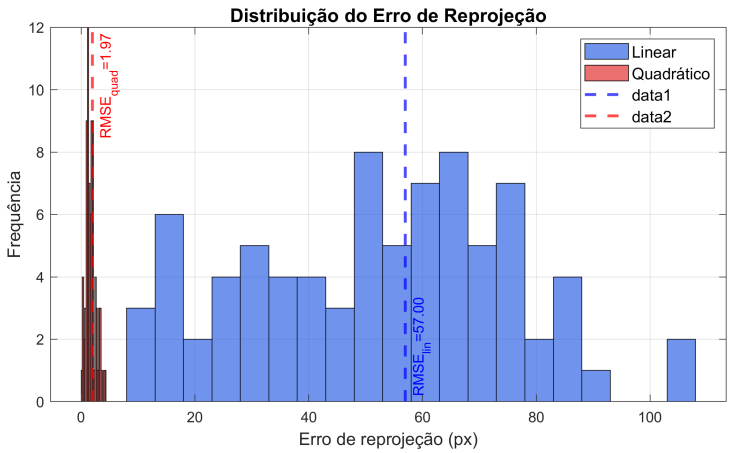

Figura 5 – Histograma do erro de reprojeção para os modelos linear (azul) e quadrático (vermelho). As linhas tracejadas marcam os valores de RMSE.

Fonte: Gerada pela autora via Script MATLAB 3.

6 FORMULAÇÃO RECURSIVA

6.1 Motivação

O método da Seção 3.3 utiliza todos os pares de calibração de uma vez para calcular $\hat{\mathbf{a}}$. Nesse sentido, a **formulação recursiva** [2] permite atualizar o estimador a cada novo par coletado, sem recalcular a pseudo-inversa completa — o que é vantajoso em sistemas com coleta incremental de dados.

6.2 Equações Recursivas

Definindo $\mathbf{P}_k = (\mathbf{X}_k^T \mathbf{X}_k)^{-1}$, a atualização a cada nova medida $(\mathbf{x}_{k+1}, y_{k+1})$ é:

$$\mathbf{K}_{k+1} = \frac{\mathbf{P}_k \mathbf{x}_{k+1}}{1 + \mathbf{x}_{k+1}^T \mathbf{P}_k \mathbf{x}_{k+1}} \tag{6.1}$$

$$\hat{\mathbf{a}}_{k+1} = \hat{\mathbf{a}}_k + \mathbf{K}_{k+1} \underbrace{\left(y_{k+1} - \mathbf{x}_{k+1}^T \hat{\mathbf{a}}_k\right)}_{\text{inovação}} \tag{6.2}$$

$$\mathbf{P}_{k+1} = \mathbf{P}_k - \mathbf{K}_{k+1} \mathbf{x}_{k+1}^T \mathbf{P}_k \tag{6.3}$$

com inicialização $\hat{\mathbf{a}}_0 = \mathbf{0}$ e $\mathbf{P}_0 = 10^3 \mathbf{I}$.

A Figura 6 mostra a convergência do estimador recursivo.

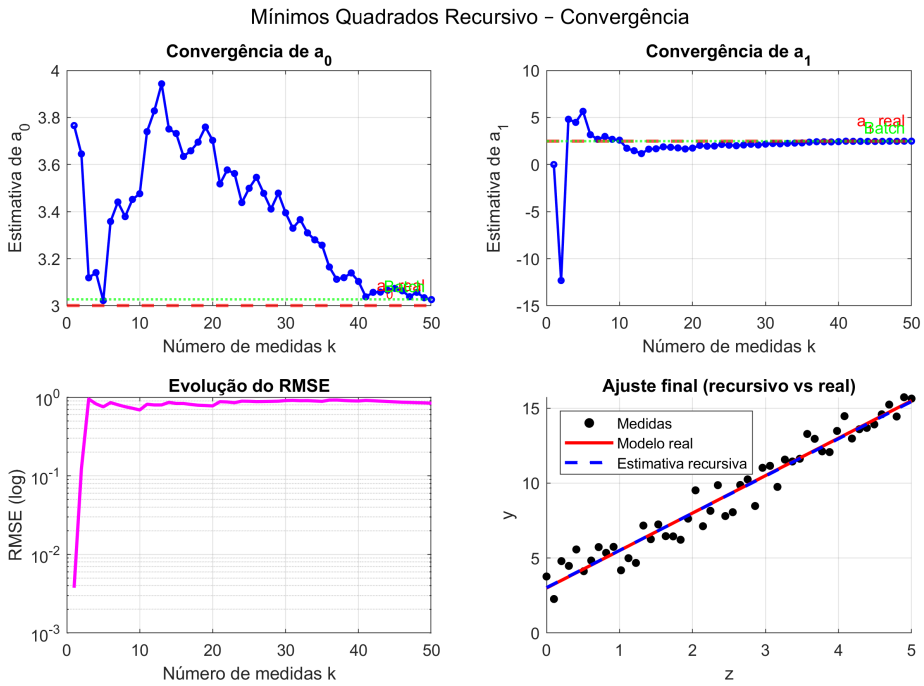

Figura 6 – Convergência do estimador recursivo para os parâmetros a_0 e a_1. Linha tracejada vermelha: valor real. Linha verde: resultado do método *batch*.

Fonte: Gerada pela autora via Script MATLAB 4.

Ponto em aberto

Questão ao orientador: A formulação recursiva (6.1)–(6.3) é equivalente ao Filtro de Kalman para estado estático. Para o problema de calibração, os parâmetros do mapeamento são suficientemente estáveis no tempo, ou variações de temperatura e iluminação exigiriam uma abordagem com estado dinâmico?

7 PRÓXIMOS PASSOS

Com base no estudo realizado, os próximos passos previstos são:

1. **Coleta de dados reais**: montar o padrão de xadrez no caixão e registrar pares Kinect↔ Projetor em diferentes alturas da areia;
2. **Validação experimental**: aplicar os scripts MATLAB aos dados reais e comparar o RMSE dos modelos linear e quadrático;
3. **Integração com o MDT**: estudar o formato dos modelos digitais de terreno das cartas topográficas de referência e implementar a comparação com a superfície lida pelo Kinect;
4. **Integração com Google Earth / OSM**: implementar a navegação geoespacial a partir de pontos indicados no caixão, usando a calibração desenvolvida para converter coordenadas do sensor em coordenadas geográficas;
5. **Escrita do capítulo de Metodologia** e do capítulo de Resultados do PFC.

REFERÊNCIAS

[1] INSTITUTO MILITAR DE ENGENHARIA. **Proposta de Projeto Final de Curso**: Caixão de Areia com Realidade Aumentada. Resende, 2025.

[2] PINHEIRO, [nome]. **Teoria dos Mínimos Quadrados** – Aula 16. Notas de aula, Controle Ótimo. [Universidade], 2021.

[3] KREYLOS, O. **Augmented Reality Sandbox (SARndbox)**. UC Davis KeckCAVES, 2012. Disponível em: https://web.cs.ucdavis.edu/~okreylos/ResDev/SARndbox/. Acesso em: fev. 2026.

[4] KECKCAVES. **SARndbox – README**. GitHub: KeckCAVES/SARndbox. Disponível em: https://github.com/KeckCAVES/SARndbox. Acesso em: fev. 2026.

[5] AR-SANDBOX.EU. **Augmented Reality Sandbox DIY Guide**. Disponível em: https://ar-sandbox.eu/augmented-reality-sandbox-diy/. Acesso em: fev. 2026.

[6] WOLF, T.; PAULSEN, R. R. **Magic-Sand: Software for Operating an Augmented Reality Sandbox**. GitHub: thomwolf/Magic-Sand. Disponível em: https://github.com/thomwolf/Magic-Sand. Acesso em: fev. 2026.

[7] HARTLEY, R.; ZISSERMAN, A. **Multiple View Geometry in Computer Vision**. 2. ed. Cambridge: Cambridge University Press, 2003.

[8] ZHANG, Z. A flexible new technique for camera calibration. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, v. 22, n. 11, p. 1330–1334, nov. 2000.